

시스템 장애 분석 및 이슈 리포트

OOM Crash · CPU Latency · Deadlock

실제 실행 로그를 근거로 재현, 원인 추론, 임시 조치와 결과를 검증했다. 제공 바이너리는 수정하거나 디컴파일하지 않았다.

실험일: 2026-09-05 / 모든 본문 시각은 KST / FAIL #15-#18 보완판

환경: Ubuntu 22.04.5 LTS, Linux 6.17.8 x86_64, 일반 사용자 analyst (UID 1000). OrbStack Docker 컨테이너에 메모리 1GiB·CPU 상한 2코어를 적용했다.

사례	주요 결과
OOM	100 → 200MB: 생존 11.412 → 23.687초. 512MB 추가 검증에서는 캐시 정리 확인.
CPU	100 → 40%: Watchdog 종료에서 75초 관측 중 생존 및 cooldown으로 전환.
Deadlock	true → false: 순환 자원 대기에서 작업 진행으로 전환.
보너스	등록된 A/B/C 순차 완료는 앱 수준의 FCFS 패턴과 부합.
평가 보완	FAIL #15: 직접 스택·LWP 추적. FAIL #18: 동시 장애 우선순위·표준 대응 절차.

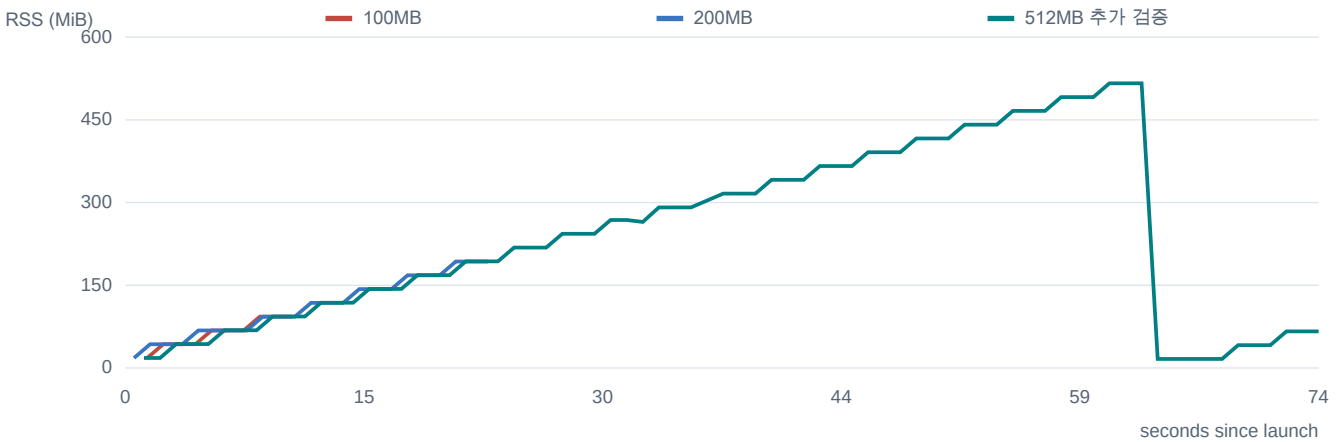
증거 읽기

본문은 필수 증거와 Before/After 비교를 포함한다. 전체 원본 로그는 이 PDF에 evidence.zip으로 첨부했다. PDF 뷰어의 첨부파일 패널에서 추출하거나 함께 제공된 프로젝트의 evidence 폴더를 열면 된다.

앱의 Heap/Current Load, OS의 RSS/CPU, 컨테이너 자원 상한은 서로 다른 지표이다. 자체 보호 종료와 실험 종료용 SIGTERM도 구분했다. 관측 기간의 회피 효과를 장기 무장애 보장으로 해석하지 않는다.

[Bug] OOM Crash - 메모리 누적 후 MemoryGuard가 SIGKILL로 자체 종료

실측 메모리 추이



약 1초 간격 monitor.sh 원본. 100/200MB는 종료 직전까지 누적, 512MB는 캐시 정리 후 RSS 감소.

1. Description (현상 설명)

2026-09-05 15:48 KST, 제공된 agent-leak-app-x86를 일반 사용자 analyst(UID 1000)로 실행했다.

MEMORY_LIMIT=100, CPU_MAX_OCCUPY=40, MULTI_THREAD_ENABLE=false 조건에서 RSS가 단계적으로 증가했고, 시작 후 11.412초에 자체 종료되었다. 메모리 한도를 200MB로 올려도 23.687초 후 같은 장애가 재현되었다.

환경은 OrbStack의 Ubuntu 22.04.5 LTS 컨테이너, Linux 6.17.8, x86_64이다. 컨테이너 메모리 제한은 1GiB, CPU 할당 상한은 2코어이며 외부 네트워크는 차단했다. 컨테이너 내부 0.0.0.0:15034 바인딩은 가능하다. 모든 부트 검사 6개가 통과했다.

프로젝트 루트에서 이미지를 빌드한 뒤 각 명령을 실행한다. 기존 증거 폴더가 있으면 실행기가 덮어쓰기를 거부하므로 재실험에는 새 이름을 사용한다.

```
docker build -t b1-2-lab .
docker run --rm --init --memory=1g --cpus=2 --network=none \
-v "$PWD:/work" b1-2-lab python3 scripts/run_case.py oom/before \
--memory 100 --cpu 40 --multi false --seconds 90
docker run --rm --init --memory=1g --cpus=2 --network=none \
-v "$PWD:/work" b1-2-lab python3 scripts/run_case.py oom/after \
--memory 200 --cpu 40 --multi false --seconds 90
```

2. Evidence & Logs (증거 자료)

실제 작업 PID는 Before 33, After 34이다. 부모 실행기 PID 14/15는 자식을 기다리므로 부모의 작은 RSS를 앱 메모리로 오인하지 않았다. 부트 로그, PID 계층, 실행 파일 경로로 대상을 확인했다.

monitor.sh 원문에서 발췌했다. RSS_KB는 KiB 단위이고 MEM은 Linux가 보고하는 전체 메모리 대비 비율이다. MEM을 앱 한도 소진율로 해석하지 않았다.

```
[2026-09-05T15:48:16+09:00] PID:33 CPU:0.0% RSS_KB:18512 MEM:0.1% STATE:SN
[2026-09-05T15:48:20+09:00] PID:33 CPU:4.0% RSS_KB:69720 MEM:0.4% STATE:SN
[2026-09-05T15:48:25+09:00] PID:33 CPU:0.0% RSS_KB:95324 MEM:0.5% STATE:SN
[2026-09-05T15:49:11+09:00] PID:34 CPU:0.0% RSS_KB:197708 MEM:1.2% STATE:SN
```

Before 종료 직전 및 터미널 출력:

```
2026-09-05 15:48:25,988 [INFO] [MemoryWorker] Current Heap: 100MB
2026-09-05 15:48:25,989 [CRITICAL] [MemoryGuard] Memory limit exceeded (100MB >= 100MB) / (Recommend Over 256MB)
2026-09-05 15:48:25,989 [CRITICAL] [MemoryGuard] Self-terminating process 33 to prevent system instability.
>>> [SYSTEM] SELF-TERMINATED (Memory Limit Exceeded) <<<
```

After에서도 15:49:12.041에 **Memory limit exceeded (200MB >= 200MB)**가 기록되었다. 두 실행 모두 Python `Popen.returncode=-9(SIGKILL)`, `observer_stopped=false`였다. 종료 후 `ps`에서 앱이 사라지고 리슨 소켓도 사라졌다. `memory.events`의 `oom`, `oom_kill`은 모두 0이다.

원본: Before 실행 로그 (`../evidence/oom/before/app.log`), After 실행 로그 (`../evidence/oom/after/app.log`), Before 관제 (`../evidence/oom/before/monitor-33.log`), After 관제 (`../evidence/oom/after/monitor-34.log`), Before `ps` (`../evidence/oom/before/ps.txt`), After `ps` (`../evidence/oom/after/ps.txt`), 실행 전 환경 (`../evidence/oom/before/preflight.txt`), 종료 후 상태 (`../evidence/oom/before/postflight.txt`).

3. Root Cause Analysis (원인 분석)

관측 사실: 앱의 Heap 카운터는 약 3.05초마다 25MB씩 늘었고 RSS도 약 25MiB씩 증가했다. 해제 없이 데이터가 누적되는 누수와 유사한 누적 패턴이다. 다만 아래 512MB 추가 검증에서 캐시 해제가 관측되어, 영구적인 미해제 결함으로 단정하지 않는다. 앱의 Heap 카운터가 설정 한도에 도달하자 MemoryGuard가 종료를 선언하고 실제 SIGKILL 종료가 뒤따랐다.

OS 원리: 프로세스의 가상 주소 공간에는 코드, 힙, 스택, 매핑 영역 등이 있다. 힙에 할당된 데이터에 접근하면 물리 페이지가 사용되며 RSS에 반영될 수 있다. RSS는 상주 페이지 전체를 포함하므로 앱의 Heap 카운터와 동일하지 않다. 증가가 계속되면 다른 프로세스가 쓸 메모리가 줄고 회수·스왑 부담이 커질 수 있다. RSS 정의는 [Linux /proc 공식 문서](#)를 참고했다.

종료 원인: 이번 장애는 애플리케이션 MemoryGuard의 메모리 보호 종료이다. 앱 로그, SIGKILL 종료 상태, cgroup OOM 카운터 0이 이를 뒷받침한다. OS 전체 메모리 고갈에 따른 OOM Killer 동작으로 분류하지 않는다.

확인 한계: 디컴파일·역공학 없이 실행 결과만 분석했다. 어떤 자료구조가 참조를 유지하는지, 의도적인 캐시인지 코드 결함인지는 확인할 수 없다. 마지막 1초 샘플 이후 메모리를 할당하고 즉시 종료하므로 관제 최대 RSS는 종료 순간의 최고 RSS가 아니다. Heap 100/200MB 로그와 93.09/193.07MiB 관제값의 차이를 모순으로 보지 않았다.

4. Workaround & Verification (조치 및 검증)

비교 항목	Before	After
MEMORY_LIMIT	100MB	200MB
나머지 기능 설정	CPU=40, MULTI=false	CPU=40, MULTI=false
시작 KST / 작업 PID	15:48:14.648 / 33	15:48:48.467 / 34
생존 시간	11.412초	23.687초
최초 → 마지막 RSS	18,512 → 95,324KiB	18,480 → 197,708KiB
종료 직전 앱 Heap	100MB	200MB
종료 결과	MemoryGuard / SIGKILL	MemoryGuard / SIGKILL
관측자가 중지했는가	아니오	아니오
결과 원본	Before (../evidence/oom/before/result.json)	After (../evidence/oom/after/result.json)

실험별 로그·카업로드 경로는 증거 격리를 위해 다른 디렉터리를 사용하고 내용·권한은 같게 유지했다. 변경한 기능 변수는 MEMORY_LIMIT 하나이다.

생존 시간이 12.275초 증가해 약 2.08배가 되었다. 같은 증가 패턴과 동일 종료는 계속되므로 한도 상향은 시간을 벌어주는 임시 조치이다. 정상화 또는 누수 해결로 보고하지 않는다. 근본 조치로 할당·참조 유지 지점의 소스 리뷰, 캐시 상한·수명 설정, 힙 프로파일링 후 장시간 RSS 안정화 검증을 제안한다.

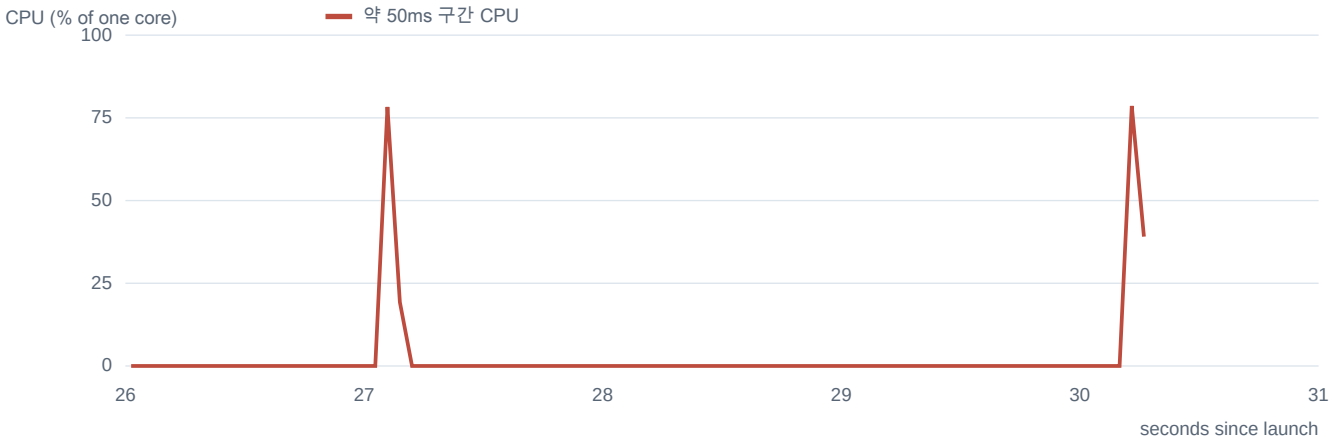
추가 검증: MEMORY_LIMIT=512에서 정리 경로 확인

같은 CPU=40, MULTI=false인 cpu/after 실행 (../evidence/cpu/after/result.json)을 MEMORY_LIMIT=512의 추가 검증으로도 활용했다. 75.093초까지 자체 종료는 없었고, 15:50:46.822에 Memory Usage Reached Limit (525MB). Starting cleanup..., 이어 Memory Cache Flushed. Process Stabilized.와 MEMORY RECOVERED (Cache Cleared)가 출력되었다. 관제 원본 (../evidence/cpu/after/monitor-2443.log)의 RSS는 15:50:46 528,672KiB에서 15:50:47 16,752KiB로 감소했다.

따라서 100→200MB는 종료 지연에 그쳤으나, 512MB 설정에서는 이 관측 구간에 정상 캐시 정리 경로가 동작했다. 낮은 한도에서 누적 중 보호 종료되는 설정 의존 동작이 직접적인 장애 원인이다. 내부 분기 기준을 역공학하지 않았으므로 모든 중간 설정의 동작이나 장기 무누수까지 일반화하지 않는다. 임계치 상향 자체가 코드를 수정한 것은 아니다.

[Bug] CPU Latency - CPU 작업 중 Watchdog가 SIGTERM 보호 종료

Watchdog 종료 직전 실제 CPU burst



PID 1089의 마지막 5초. 최대 78.6%는 약 50ms 평균이다. 앱 내부 Load 52.69% 및 1초 관제 최대 5.0%와 구분한다.

1. Description (현상 설명)

2026-09-05 15:49:12.258 KST에 `MEMORY_LIMIT=512`, `CPU_MAX_OCCUPY=100`, `MULTI_THREAD_ENABLE=false`로 실행한 앱에서 내부 `Current Load`가 5.00%에서 52.69%로 증가했다. 30.504초 후 Watchdog가 SIGTERM 종료를 실행했다. 실제 작업 PID는 1089이며 부모 실행기는 1069이다.

환경: Ubuntu 22.04.5 / Linux 6.17.8 x86_64, UID 1000, 컨테이너 메모리 1GiB·CPU 상한 2코어·외부 네트워크 없음. 필수 경로·카·포트·쓰기 권한의 부트 검사가 모두 통과했다.

```
docker run --rm --init --memory=1g --cpus=2 --network=none \
-v "$PWD:/work" b1-2-lab python3 scripts/run_case.py cpu/before-02 \
--memory 512 --cpu 100 --multi false --seconds 100
docker run --rm --init --memory=1g --cpus=2 --network=none \
-v "$PWD:/work" b1-2-lab python3 scripts/run_case.py cpu/after \
--memory 512 --cpu 40 --multi false --seconds 75
```

측정 범위: 특정 프로세스의 짧은 CPU 상승과 보호 종료를 확인했다. 별도 클라이언트의 응답 시간을 측정하지 않았으므로 실제 서비스 지연량이나 시스템 전체 CPU 포화를 주장하지 않는다.

2. Evidence & Logs (증거 자료)

Before 앱 원문:

```
2026-09-05 15:49:14,389 [INFO] [CpuWorker] Current Load: 5.00%
2026-09-05 15:49:39,324 [INFO] [CpuWorker] Current Load: 48.91%
2026-09-05 15:49:42,442 [INFO] [CpuWorker] Current Load: 52.69%
2026-09-05 15:49:42,544 [CRITICAL] [CpuWorker] CPU Threshold Violated! (52.690000000000005%).
>>> [SYSTEM] WATCHDOG: INITIATING EMERGENCY ABORT (SIGTERM) <<<
```

1초 간격 `monitor.sh`에서는 PID 1089가 0.0~5.0%로 관측되었다. 짧은 연산을 1초 평균만으로 판단하지 않기 위해 `/proc/PID/stat`의 `utime+stime` 차이를 약 50ms 간격으로 추가 수집했다. CPU는 코어 하나를 100%로 계산한다. 다음 CSV 원문은 UTC이며 KST는 +9시간이다.

```
2026-09-05T06:49:42.373+00:00,1089,0.0,0.051495,S,16488
2026-09-05T06:49:42.424+00:00,1089,0.0,0.050968,S,16488
2026-09-05T06:49:42.475+00:00,1089,78.6,0.050898,R,16488
2026-09-05T06:49:42.526+00:00,1089,39.1,0.051105,S,16488
```

열은 `timestamp,pid,cpu_percent,interval_seconds,state,rss_kb`이다. 0%에서 78.6%로 급상승한 구간에 R(실행/실행 가능) 상태를 관측했고, 직후 앱이 임계치 위반을 기록했다. 약 10ms 단위 CPU tick을 50ms 구간에서 계산하므로 단일 샘플의 정밀도에는 한계가 있다.

원본: 앱 Before (`../evidence/cpu/before-02/app.log`), 앱 After (`../evidence/cpu/after/app.log`), 관제 Before (`../evidence/cpu/before-02/monitor-1089.log`), 관제 After (`../evidence/cpu/after/monitor-2443.log`), 50ms Before (`../evidence/cpu/before-02/cpu-burst-1089.csv`), 50ms After (`../evidence/cpu/after/cpu-burst-2443.csv`), ps (`../evidence/cpu/before-02/ps.txt`), top (`../evidence/cpu/before-02/top.txt`).

3. Root Cause Analysis (원인 분석)

직접 원인: CpuWorker가 부하 증가를 기록하다 내부 정책상 허용 범위를 위반했고, Watchdog가 비상 중단을 선언했다. Python 종료 상태 -15, `observer_stopped=false`, 종료 후 PID-소켓 소멸이 보호 종료를 뒷받침한다. 일반 예외의 스택 트레이스나 관측자의 강제 중단이 원인이라는 증거는 없다.

설정 해석: CPU_MAX_OCCUPY=100을 “실측 CPU가 100%가 되어야 종료된다”로 해석하면 안 된다. 실제 종료 로그의 내부 값은 52.69%였고, 부트 화면에는 `Recommend Under 50%`가 표시되었다. CPU_MAX_OCCUPY는 앱의 작업 부하 동작에 영향을 주지만, 관측만으로 Watchdog의 내부 비교식 전체를 확정할 수 없다. 40으로 변경한 실행에서는 40.00% 도달 시 `cooldown`이 시작되어 종료를 회피했다.

OS 원리: 실행 가능한 작업들은 CPU 실행 시간을 경쟁한다. CPU를 오래 사용하는 작업이 증가하면 다른 작업의 실행 대기가 길어질 수 있다. 스케줄러의 nice 우선순위 조정은 실행 시간 배분에 영향을 주지만 고정된 CPU 사용률 상한을 보장하지 않는다. 앱은 `nice=10`을 기록했다. [Linux 스케줄러 공식 문서](#)를 참고했다.

측정 해석과 한계: 앱의 `Current Load`는 OS 실측 CPU와 다른 지표이다. 52.69%를 시스템 CPU 사용률로 옮겨 적지 않았다. 1초 관제, 50ms 관제, ps의 실행 기간 평균도 서로 다른 시간 창이다. After는 MemoryWorker까지 작동하므로 실제 CPU 최대치가 오히려 커졌다. 따라서 “실측 CPU가 낮아졌다”가 아니라 “작업 정책이 `cooldown`으로 전환되고 관측 중 자체 종료가 사라졌다”가 검증 결과다.

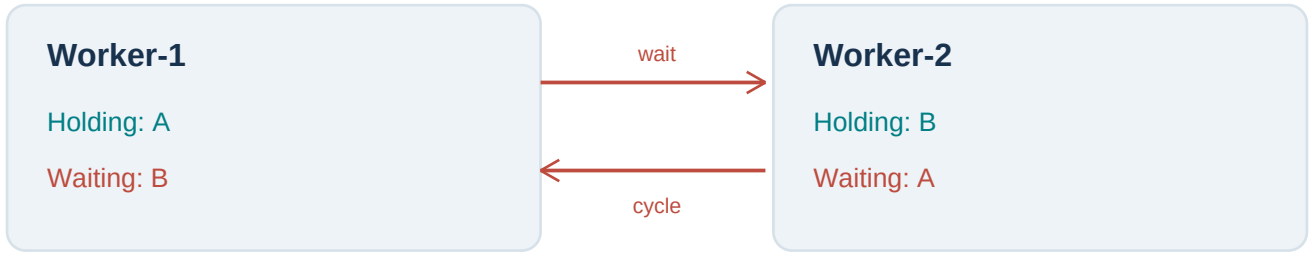
4. Workaround & Verification (조치 및 검증)

비교 항목	Before	After
CPU_MAX_OCCUPY	100%	40%
고정 기능 설정	MEMORY=512, MULTI=false	MEMORY=512, MULTI=false
시작 KST / 작업 PID	15:49:12.258 / 1089	15:49:42.848 / 2443
생존 / 관측 시간	30.504초 후 자체 종료	75.093초까지 생존
앱 내부 Load	5.00 → 52.69%	40.00%에서 cooldown
1초 실측 CPU 최대	5.0%	7.9%
약 50ms 실측 CPU 최대	78.6%	99.2%
종료 주체	Watchdog / SIGTERM	관측자 / SIGTERM
observer_stopped	false	true
결과 원본	Before (../evidence/cpu/before-02/result.json)	After (../evidence/cpu/after/result.json)

After 15:50:12.842에 Peak reached (40.00%). Starting cooldown..., 15:50:37.801에 Cooldown complete (5.00%). Resuming load increase...를 확인했다. Before 종료 시점을 넘겨 약 2.46배 길게 관측했으며 Watchdog 종료는 없었다. 동일한 -15라도 After는 실험 종료를 위해 관측자가 보낸 신호이므로 장애로 계산하지 않는다.

기능 변수 하나만 변경했고 실험별 디렉터리는 격리했다. 부하 값의 증가 폭에는 실행 간 차이가 있으므로 단일 쌍의 생존 시간을 보장값으로 사용하지 않는다. 정상화 설정은 MEMORY_LIMIT=512, CPU_MAX_OCCUPY=40, MULTI_THREAD_ENABLE=false이다. 실제 운영 성능 검증에는 요청 지연·처리량 및 더 긴 관측이 추가로 필요하다.

[Bug] Deadlock - 두 워커의 반대 락 획득 순서로 작업 진행 정지



A = Shared_Memory_A, B = Socket_Pool_B. 로그의 락 보유·요청 관계를 재구성한 도식이다.

1. Description (현상 설명)

2026-09-05 15:50:58.028 KST, MEMORY_LIMIT=512, CPU_MAX_OCCUPY=40, MULTI_THREAD_ENABLE=true로 실행했다. 약 9초 뒤 두 워커가 WAITING ... BLOCKED를 기록한 후 로그 증가와 작업 진행이 멈췄다. 실제 작업 PID 5782는 약 60초 관측이 끝날 때까지 존재했다. 예외 종료가 아니라 살아 있는 프로세스의 교착상태이다.

Ubuntu 22.04.5 / Linux 6.17.8 x86_64 컨테이너에서 UID 1000으로 실행했다. 메모리 상한 1GiB·CPU 상한 2코어·외부 네트워크 없음이며, 필수 경로·카 권한·15034 포트 부트 검사는 모두 통과했다.

```
docker run --rm --init --memory=1g --cpus=2 --network=none \
-v "$PWD:/work" b1-2-lab python3 scripts/run_case.py deadlock/before \
--memory 512 --cpu 40 --multi true --seconds 60
docker run --rm --init --memory=1g --cpus=2 --network=none \
-v "$PWD:/work" b1-2-lab python3 scripts/run_case.py deadlock/after \
--memory 512 --cpu 40 --multi false --seconds 60
```

2. Evidence & Logs (증거 자료)

15:51:53 KST 스냅샷의 PID 존재 증거:

```
$ ps -ef | grep '[a]gent-leak-app'
analyst      5763      5756  0 15:50 ?        00:00:00 /work/agent-app-leak/agent-leak-app-x86
analyst      5782      5763  0 15:50 ?        00:00:00 /work/agent-app-leak/agent-leak-app-x86
```

같은 시점 `ps -L -p 5763,5782 -o pid,ppid,lwp,nlwp,stat,pcpu,pmem,rss,time,wchan:32,comm`에서 작업 PID의 세 LWP는 5782, 6079, 6080이었다. 모두 SNI, CPU 0.0%, TIME 00:00:00, `futex_wait`였다. 부모 5763은 `do_wait`로 자식을 기다렸다. 스레드별 RSS는 같은 주소 공간의 값이므로 세 번 더하지 않는다.

monitor.sh 원문 발췌:

```
[2026-09-05T15:51:08+09:00] PID:5782 CPU:0.0% RSS_KB:18556 MEM:0.1% STATE:SNL
[2026-09-05T15:51:25+09:00] PID:5782 CPU:0.0% RSS_KB:18556 MEM:0.1% STATE:SNL
[2026-09-05T15:51:45+09:00] PID:5782 CPU:0.0% RSS_KB:18556 MEM:0.1% STATE:SNL
[2026-09-05T15:51:57+09:00] PID:5782 CPU:0.0% RSS_KB:12300 MEM:0.0% STATE:SNL
```

전체 58개 관제 샘플에서 CPU는 0.0%였다. 15:51:08~45의 RSS는 18,556KiB로 일정했다. 이후 RSS가 12,300KiB로 감소했으므로 전체 구간의 메모리가 완전히 불변했다고 쓰지 않는다. 감소 원인은 이 증거만으로 확정할 수 없고, 감소 후에도 로그·스레드 대기는 해소되지 않았다.

마지막 두 로그는 15:51:07.202에 기록되었다. 아래는 같은 실행의 핵심 메시지 부분만 발췌한 것이다.

```
[Worker-Thread-1] LOCK ACQUIRED: [Shared_Memory_A]. (Holding...)
[Worker-Thread-2] LOCK ACQUIRED: [Socket_Pool_B]. (Holding...)
[Worker-Thread-1] WAITING for [Socket_Pool_B]... (Status: BLOCKED)
[Worker-Thread-2] WAITING for [Shared_Memory_A]... (Status: BLOCKED)
```

그 뒤 약 51초 동안 새 로그가 없었다. 반복 ps.txt 스냅샷에서 app.log bytes=2657이 유지되었다. 재시작 전에 PID·관제·스레드·로그 증거를 확보했다.

원본: 실행 로그 (../evidence/deadlock/before/app.log), 관제 (../evidence/deadlock/before/monitor-5782.log), PID·스레드·로그 크기 (../evidence/deadlock/before/ps.txt), top -H (../evidence/deadlock/before/top.txt), Before 결과 (../evidence/deadlock/before/result.json).

추가 직접 증거: GDB 스택 + strace의 워커/LWP 연결 (FAIL #15)

2026-09-05 16:28:13~16:29:13 KST에 동일 기능 설정(512/40/true)으로 별도 진단 실행을 수행했다. 원본 Before/After 수치는 변경하지 않았다. 추가 실행의 실제 작업 PID는 35, 부모는 16이며 앱은 기존과 같이 analyst(UID 1000)로 실행했다. 진단 전용 컨테이너에 GDB 12.1, strace 5.16을 추가하고 추적 도구만 root로 실행했다.

자료	KST 시간 / 대상	원본
시스템 호출 추적	16:28:14.039~26.101 / PID 35와 자식 LWP	strace-futex-write.txt (../evidence/deadlock/diagnostic-01/strace-futex-write.txt)
전체 스레드 스택 1	16:28:26.101 / LWP 35, 331, 332	gdb-thread-bt-01.txt (../evidence/deadlock/diagnostic-01/gdb-thread-bt-01.txt)
전체 스레드 스택 2	16:28:41.242 / 같은 LWP	gdb-thread-bt-02.txt (../evidence/deadlock/diagnostic-01/gdb-thread-bt-02.txt)
디버거 분리 후 대기	16:28:26.220 및 41.360 / 같은 LWP	proc 1 (../evidence/deadlock/diagnostic-01/proc-after-gdb-01.txt), proc 2 (../evidence/deadlock/diagnostic-01/proc-after-gdb-02.txt)
앱·환경·종료 기록	60초 관측, 관측자 SIGTERM	앱 로그 (../evidence/deadlock/diagnostic-01/app.log), 환경 (../evidence/deadlock/diagnostic-01/preflight.txt), 결과 (../evidence/deadlock/diagnostic-01/result.json), 수집 시각·명령 (../evidence/deadlock/diagnostic-01/diagnostic-events.json)

이름과 LWP의 직접 연결: strace는 각 write()를 호출한 Linux TID를 행 맨 앞에 표시한다. 16:28:23.003206의 TID 331이 Worker-Thread-1의 WAITING for [Socket_Pool_B]를 stdout에 썼고, 16:28:23.003704의 TID 332가 Worker-Thread-2의 WAITING for [Shared_Memory_A]를 썼다. 같은 TID의 앞선 LOCK ACQUIRED 출력까지 연결하면 다음 관계가 성립한다. OS comm은 둘 다 잘린 실행 파일명이라 comm만으로 이름을 추정하지 않았다.

앱 이름 / LWP	로그의 보유 자원	다음 요청 자원	마지막 지속 futex 대기 주소
Worker-Thread-1 / 331	Shared_Memory_A	Socket_Pool_B	0x1214e2b0
Worker-Thread-2 / 332	Socket_Pool_B	Shared_Memory_A	0x12154e90

strace의 마지막 대기 호출 원문:

```
331 16:28:23.003371 futex(0x1214e2b0, FUTEX_WAIT_BITSET_PRIVATE|FUTEX_CLOCK_REALTIME, 0, NULL, FUTEX_BITSET_MATCH_ANY <unfinished ...>
332 16:28:23.003865 futex(0x12154e90, FUTEX_WAIT_BITSET_PRIVATE|FUTEX_CLOCK_REALTIME, 0, NULL, FUTEX_BITSET_MATCH_ANY <detached ...>
```

NULL은 이 futex 대기 호출에 제한 시간이 전달되지 않았음을 나타낸다. <detached ...>는 수집기가 strace에 SIGINT를 보내 분리한 표기이며, 앱의 락 획득 성공 또는 종료가 아니다. 추적 중 나타나는 다른 futex 주소에는 로깅·런타임 동기화도 포함될 수 있어 모든 futex를 업무 자원 락으로 분류하지 않았다.

GDB 콜스택 원문 발췌: 1차 캡처에서 각 워커의 #0~#2 프레임은 아래와 같다. 두 번째 캡처에도 같은 PC와 PyThread_acquire_lock_timed 프레임이 유지되었다. 전체 12개 프레임/스레드는 위 원본에 보존했다.

```
Thread 3 (Thread 0x7fe791277640 (LWP 332) "agent-leak-app-"):
#0  0x00007fe792c9b0d7 in ?? () from /lib/x86_64-linux-gnu/libc.so.6
#1  0x00007fe792ca6c38 in ?? () from /lib/x86_64-linux-gnu/libc.so.6
#2  0x00007fe79284ecec in PyThread_acquire_lock_timed () from /tmp/_MEILP5dBf/libpython3.10.so.1.0
Thread 2 (Thread 0x7fe791a78640 (LWP 331) "agent-leak-app-"):
#0  0x00007fe792c9b0d7 in ?? () from /lib/x86_64-linux-gnu/libc.so.6
#1  0x00007fe792ca6c38 in ?? () from /lib/x86_64-linux-gnu/libc.so.6
#2  0x00007fe79284ecec in PyThread_acquire_lock_timed () from /tmp/_MEILP5dBf/libpython3.10.so.1.0
```

두 GDB 캡처 사이 시작 시각 차이는 15.141초다. GDB를 분리한 직후 /proc/35/task/331/syscall은 두 번 모두 202 0x1214e2b0 0x189 ..., LWP 332는 202 0x12154e90 0x189 ...를 나타냈다. 이 x86_64 환경의 syscall 202를 strace가 futex로 해석한 결과와 일치한다. 두 LWP의 wchan은 __futex_wait, 앱 로그 크기는 모두 2,664바이트였다. 즉 디버거가 멈춘 순간만의 상태가 아니라, 디버거 분리 후에도 같은 자원 대기가 지속되었다.

수집 명령과 재현: 기존 이미지를 빌드하고 진단 이미지를 추가한다. 마지막 인자는 새 증거 폴더명으로 바꿔 재실험할 수 있다.

```
docker build -t b1-2-lab .
docker build -f Dockerfile.diagnostics -t b1-2-diagnostics .
docker run --rm --init --memory=1g --cpus=2 --network=none \
  --cap-add=SYS_PTRACE --security-opt=seccomp=unconfined --user root \
  -v "$PWD:/work" b1-2-diagnostics \
  python3 scripts/capture_deadlock.py deadlock/diagnostic-rerun-01
```

수집기는 runuser -u analyst로 기존 실험 실행기를 시작한 뒤, 작업 자식에 strace -f -tt -T -s 512 -e trace=futex,write -p PID를 붙인다. BLOCKED 두 건 확인 후 strace를 먼저 분리하고, gdb -q -nx -batch -ex 'info threads' -ex 'thread apply all bt 12' -ex detach -p PID로 두 번 스택을 수집한다. 실제 옵션 전체와 UTC 시각은 diagnostic-events.json에 남겼다. 앱 실행 파일의 해시는 최초 실험과 같다.

영향과 확인 범위: GDB는 attach 시 잠깐 앱을 정지시키며 strace도 실행 타이밍에 영향을 준다. 따라서 추가 실행을 기존 생존 시간 통계에 섞지 않았다. 메모리 쓰기, 함수 호출 주입, 락 강제 해제, 디스어셈블·디컴파일은 수행하지 않았다. 바이너리에 디버그 정보가 없어 일부 네이티브 프레임은 ??이며 Python 소스 줄 번호는 복원하지 않았다. 그럼에도 로그를 쓴 실제 워커 TID → 무기한 futex 대기 → Python 락 획득 콜스택 → 시간 경과 후 같은 대기 유지를 직접 증거로 연결했다. A/B의 의미와 소유 관계는 앱 로그에 근거하며 락 객체 내부의 소유자 필드를 읽어낸 것은 아니다.

명령 의미 참고: GDB 전체 스레드 backtrace 공식 문서, strace 공식 사용 안내.

3. Root Cause Analysis (원인 분석)

로그가 나타내는 대기 관계는 다음과 같다. 화살표는 해당 자원 또는 작업의 완료를 기다린다는 뜻이다.

```
Worker-1 holds A; waits for B held by Worker-2
Worker-2 holds B; waits for A held by Worker-1
Worker-1 -> B -> Worker-2 -> A -> Worker-1
```

교착상태의 네 조건이 모두 성립하는 패턴이다.

조건	이 실행에서의 근거
상호 배제	A/B 각각 LOCK ACQUIRED와 상대 워커 BLOCKED
점유 대기	A 또는 B를 Holding한 채 두 번째 자원을 요청
비선점	상대 소유 자원이 반환되지 않고 대기 지속; 강제 회수 로그 없음
순환 대기	Worker-1 → B → Worker-2 → A → Worker-1

정상적인 휴면 상태도 CPU가 0이고 `futex_wait`일 수 있다. 이번에는 서로의 자원을 기다리는 명시적 로그, 세 LWP의 대기, 장시간 작업 로그 정지가 함께 있으므로 순환 대기에 따른 교착상태로 판단한다. 메인 스레드는 `Waiting for worker threads to complete transactions...` 후 기다리므로 워커 완료 대기과 부합한다. 최초 Before 실행은 이름/LWP 매핑이 없었지만, 추가 진단 실행에서는 `strace`의 `write` 호출로 Worker-1/2와 LWP 331/332를 직접 연결했다.

OS는 락을 기다리는 스레드를 재우므로 CPU를 계속 소비하지 않아도 프로세스가 멈출 수 있다. 다른 스레드가 락을 해제할 때까지 `acquire`가 대기하는 일반 원리는 [Python threading 공식 문서](#)를 참고했다. 최초 분석은 로그와 `ps`에 근거했고, 추가 진단에서 GDB 콜스택·`strace`·분리 후 `syscall` 상태를 확보해 실제 락 획득 대기까지 확인했다. 내부 락 소유자 필드나 소스 코드는 조사하지 않았다.

4. Workaround & Verification (조치 및 검증)

비교 항목	Before	After
MULTI_THREAD_ENABLE	true	false
고정 기능 설정	MEMORY=512, CPU=40	MEMORY=512, CPU=40
시작 KST / 작업 PID	15:50:58.028 / 5782	15:51:58.179 / 8441
관측 시간	60.063초	60.356초
마지막 앱 로그	15:51:07.202 BLOCKED	15:52:57.267 Current Load
CPU 1초 샘플	모두 0.0%	0.0~7.9%
RSS	18,556KiB 정체 구간	18,372 → 503,376KiB
진행 확인	로그 2,657바이트로 정지	A/B/C 완료 후 Heap/Load 지속
관측 종료	관측자 SIGTERM	관측자 SIGTERM
결과 원본	Before (../evidence/deadlock/before/result.json)	After (../evidence/deadlock/after/result.json)

After에는 **Healthy System Monitoring, All tasks completed**. 이후 MemoryWorker/CpuWorker 로그가 계속 나왔고, 관측 중 **WAITING ... BLOCKED**는 없었다. After 로그 (../evidence/deadlock/after/app.log), After ps (../evidence/deadlock/after/ps.txt), After 관제 (../evidence/deadlock/after/monitor-8441.log)에서 확인한다.

MULTI_THREAD_ENABLE=false는 문제의 동시 트랜잭션 경로를 회피하는 조치다. false에서도 정상 MemoryWorker/CpuWorker를 포함해 3 LWP가 관측되므로 앱 전체가 단일 OS 스레드가 되었다고 해석하지 않는다. 두 실험 모두 관측자가 종료했고 그 이전에 자체 종료는 없었다. 60초 관측은 영구 무교착 보장이 아니다.

근본 개선은 두 경로의 락 획득 순서를 A→B로 통일하여 순환 대기 조건을 없애는 것이다. 추가로 락 획득 timeout과 실패 시 이미 잡은 락 반환을 설계하고, 반복 경쟁 상황에서도 트랜잭션이 완료되는지 검증할 수 있다.

[Runbook] 동시 장애 우선순위와 표준 대응 절차

평가 FAIL #18 보완. 적용 대상은 agent-leak-app의 메모리 보호 종료, CPU Watchdog 종료, 교착상태가 같은 시각에 보고되는 상황이다. 아래 수치·응답 시간은 이 과제를 위한 운영 판단 기준 제안이며, 체결된 SLA나 실제 동시 장애 실험 결과가 아니다. 단일 프로세스에서 세 장애가 동시에 재현되었다고 주장하지 않는다.

1. 접수와 우선순위 결정

담당자는 최초 알림을 접수하면 시각·서비스·호스트/컨테이너·PID·프로세스 시작 시각·증상을 기록한다. 같은 PID 숫자라도 시작 시각이 다르면 재시작된 별개 실행이다. 종료 전 CPU 알림과 재시작 후 교착 알림을 한 프로세스의 동시 상태로 합치지 않는다. 앱 내부 Heap/Current Load, 프로세스 RSS/CPU, cgroup 및 호스트 사용량도 별도 필드로 기록한다.

판단 순서: 영향 범위 → 현재 자원 고갈 위험 → 사용자 작업 정지 → 복구까지 남은 시간. 장애 유형 자체에 고정 순위를 부여하지 않는다.

등급	판정 기준	착수·재판단 목표
P0: 확산 위험	공용 호스트의 다른 서비스까지 실패하거나, cgroup/호스트 메모리가 계속 증가해 가용 한도 도달 예상이 30초 이내. OOM kill 반복·스왑/메모리 압력도 함께 확인	즉시 착수, 1분 이내 담당자 지정·격리 판단. 1분마다 재판단
P1: 서비스 중단	PID는 있으나 30초 이상 업무 진행이 없고 순환 대기 증거가 있음, 또는 Watchdog/MemoryGuard 종료가 반복되어 요청을 처리할 인스턴스가 없음. 공용 자원 고갈은 없음	5분 이내 착수, 15분 이내 임시 복구 또는 복구 지연 사유 기록. 5분마다 재판단
P2: 제한적 영향	한 인스턴스가 보호 종료했으나 나머지가 처리 중이고 현재 자원·응답이 안정적이거나, 짧은 CPU burst만 있고 작업은 진행됨	30분 이내 분석 착수. 악화하면 즉시 P1/P0로 승격

메모리 90% 이상은 조사 트리거다. `memory.max`가 유한한 cgroup에서는 $(\text{memory.max} - \text{memory.current}) / \text{최근 증가속도}$ 로 소진 시간을 추정한다. 증가속도가 0 이하이면 이 계산을 사용하지 않는다. 이 추정은 캐시 회수·추가 부하로 달라질 수 있으므로 `memory.events`, 호스트 MemAvailable, 최근 업무 실패를 함께 본다. 앱 MEMORY_LIMIT와 cgroup `memory.max`를 섞어 계산하지 않는다.

CPU는 OS 실행 프로세스 CPU가 1코어 기준 90% 이상으로 10초 지속되고 응답/작업 지연이 함께 발생하면 우선 조사한다. 다중 코어·스레드 환경에서는 코어 수와 cgroup CPU 할당량도 확인한다. 이 값은 앱 Watchdog 임계치가 아니며 앱 내부 Current Load로 대신 판정하지 않는다. 낮은 CPU에서도 서비스가 멈췄으면 P1 판단을 진행한다.

2. 동시 알림의 처리 순서

동시 관측 상황	첫 조치와 그 이유	다음 조치
메모리 소진 임박 + CPU 급등	P0 메모리 발생 인스턴스의 신규 작업 유입을 차단하고 격리. 공용 자원이 소진될 시간부터 확보	압력이 내려가면 CPU 실측·Watchdog 로그 확인 후 해당 인스턴스 부하 제한/설정 수정
교착으로 업무 중단 + CPU burst, 호스트 안정	P1 교착 인스턴스의 PID·락 로그·스택을 먼저 보존하고 대체 인스턴스로 전환. 짧은 burst만으로 이를 뒤로 미루지 않음	CPU가 지속 부하인지 확인하고 별도 교정
교착 + 다른 인스턴스가 호스트 CPU를 지속 독점	다른 서비스까지 지연되면 P0 CPU 인스턴스의 유입/실행 자원을 먼저 제한	확보된 여유에서 교착 스택 수집·인스턴스 교체
과거 MemoryGuard 종료 알림 + 현재 교착	이미 종료된 PID에 kill을 보내지 않고 해당 로그를 보존. 현재 요청 처리를 막는 교착을 우선 해결	새 실행 ID별로 원인을 분리하고 메모리 재발 위험 평가

같은 등급이면 영향을 받는 사용자·서비스 수가 큰 사건, 고갈 예상 시간이 짧은 사건, 복구 수단이 없는 사건 순으로 처리한다. 총괄 담당자 한 명이 순서를 기록하고, 추가 담당자가 있으면 다른 인스턴스의 증거 수집을 병행한다. 담당자가 한 명이면 P0 격리 → P1 업무 복구 → 나머지 원인 분석 순서로 진행한다. 한 인스턴스에서 나온 여러 알림이면 실행 ID와 타임라인을 먼저 대조해 중복 대응을 방지한다.

3. 표준 운영 절차

단계 A. 5초 안에 최소 상태를 확보

실제 작업 자식 PID를 `/proc/PID/exe`와 부모/자식 관계로 확인한다. 아래 명령은 이미 확인한 PID와 대상 컨테이너 안에서 실행하며, 출력은 사건별 새 디렉터리에 보존한다. 프로세스 시작 정보도 남겨 PID 재사용을 구분한다.

```
# Use the verified workload PID and a new evidence directory.
mkdir -p "$INCIDENT_DIR"
{
  date -Iseconds
  readlink "/proc/$PID/exe"
  ps -p "$PID" -o pid,ppid,lstart,stat,pcpu,rss,args
  ps -L -p "$PID" -o pid,lwp,stat,pcpu,rss,wchan:32,comm
  cat "/proc/$PID/stat"
  cat /sys/fs/cgroup/memory.current /sys/fs/cgroup/memory.max
  cat /sys/fs/cgroup/memory.events
} > "$INCIDENT_DIR/triage.txt" 2>&1
MONITOR_LOG_FILE="$INCIDENT_DIR/monitor.log" ./monitor.sh "$PID"
```

호스트에서 조회할 때는 `/proc/PID/cgroup`으로 대상 cgroup 경로를 찾는다. 현재 셸의 cgroup을 대상 앱의 것으로 가정하지 않는다. 프로세스가 이미 종료됐으면 남은 앱 로그, 감독 프로세스의 종료 상태, cgroup 카운터를 먼저 보존한다. -9/-15 만으로 OOM Killer 또는 Watchdog를 단정하지 않는다.

단계 B. 확산 차단 여부를 먼저 결정

P0에서는 긴 스택 수집을 기다리지 않는다. 가능한 최소 로그를 남긴 뒤 문제 인스턴스의 신규 요청/작업 유입을 차단하고 정상 인스턴스로 전환한다. 여유가 없으면 확인한 대상 프로세스를 중단해 자원부터 회수한다. P1/P2이고 자원이 안정적이면 앱 로그 복사, 5초 간격 `ps/top`, 1초 관제와 스택 등 증거를 먼저 확보한다. 증거 확보 지연이 위험해 중단했다면 생략한 자료와 이유를 기록한다.

단계 C. 유형별 증거와 임시 조치

유형	종료·재시작 전 핵심 증거	조치
MemoryGuard	RSS 증가, Memory limit exceeded, SELF-TERMINATED, 감독 프로세스 종료 상태, memory.events	용량 여유를 확인한 뒤 MEMORY_LIMIT=512로 재실행. 100→200은 지연에 그쳤고 512에서는 정리를 관측했다는 기존 실험 결과를 적용
CPU Watchdog	대상 PID의 OS CPU 시계열, Current Load 별도 기록, WATCHDOG SIGTERM 로그	CPU_MAX_OCCUPY=40으로 재실행하고 cooldown 확인. 필요하면 유입 제한; CPU 상한 조정만으로 코드 결함이 해결됐다고 판단하지 않음
Deadlock	PID-LWP, 마지막 락 보유/대기 로그, GDB 전체 스레드 bt, strace futex/write	증거 확보 후 MULTI_THREAD_ENABLE=false로 교체 실행. 기존 멈춘 프로세스에 export만 해서는 설정이 반영되지 않음

교착 진단은 직접 증거 및 수집 절차 (deadlock.md)의 [scripts/capture_deadlock.py](#)를 격리된 재현 환경에서 사용한다. GDB attach는 앱을 잠깐 멈추므로 원인 분류 시 부작용을 기록한다. [strace](#)를 detach한 뒤 GDB를 붙인다. 이 수집 때문에 P0 격리를 지연하지 않는다.

단계 D. 종료·재시작·재부팅의 선택

정상 인스턴스로 전환한 뒤 확인한 대상에 SIGTERM을 보내고 5초 동안 종료·PID·포트 상태를 확인한다. 남아 있으면 실행 파일과 저장한 시작 식별자가 같은 대상인지 다시 확인한 뒤 해당 PID에만 SIGKILL을 사용한다. 이름 전체를 대상으로 하는 [pkill](#), 공용 호스트 전체 재부팅은 기본 대응으로 삼지 않는다. SIGKILL은 정리 핸들러를 건너뛰므로 미완료 작업의 재처리 여부를 기록한다.

프로세스가 사라지고 15034 포트가 해제된 후 기존 필수 경로·키·권한을 유지하며 MEMORY_LIMIT=512, CPU_MAX_OCCUPY=40, MULTI_THREAD_ENABLE=false로 새 프로세스를 시작한다. 시작 시각과 새 PID를 별도로 기록한다. 이 조합은 기존 정상 경로 관측에 근거한 임시 설정이며 모든 환경의 무장해 보장이 아니다.

재부팅은 프로세스 종료나 컨테이너 교체로 복구되지 않고 호스트 자체가 응답하지 않아 서비스 복구가 불가능한 경우의 마지막 수단이다. 가능한 증거·시각·실패한 복구 시도·재부팅 판단 이유를 남긴다. 앱 한 개의 교착이나 자체 보호 종료만으로 호스트를 재부팅하지 않는다.

4. 복구 확인과 종료 기준

담당자는 새 PID 존재와 포트 LISTEN만으로 복구를 선언하지 않는다. 다음 항목을 최소 3분 확인한 뒤 결과를 사건 기록에 남긴다. 이 기간은 제안된 운영 검증 절차이며 이번 보완에서 3분 복구 실험을 추가했다는 뜻이 아니다.

- 대표 업무/테스트 작업이 실제 완료되고 로그가 계속 증가한다. Scheduler의 작업 완료와 이후 워커 진행을 확인한다.
- Memory Cache Flushed 후 RSS 감소, CPU cooldown 진입·복귀가 확인되고 새 MemoryGuard/Watchdog 종료가 없다. 주기가 길어 정리·cooldown이 아직 없으면 관측을 연장한다.
- 두 번 이상의 자원 정리 주기 및 CPU 부하 주기를 확인하고, cgroup OOM 카운터와 공용 호스트의 다른 서비스가 안정적이다.
- BLOCKED 로그와 동일 자원의 순환 대기가 재발하지 않으며, 실제 사용자 영향이 해소되었다. 단순 futex_wait 하나는 교착 재발 근거로 쓰지 않는다.

재발하면 사건을 닫지 않고 해당 인스턴스를 다시 격리하고 P0/P1 기준을 재평가한다. 반복 재시작으로 증거를 덮어쓰지 않는다. 이미 안정성이 확인된 대체 인스턴스를 유지하고, 변경한 설정·부하 조건을 개발 담당자에게 인계한다.

사건 기록에는 접수 시각, 실행 ID, 동시 알림 목록, 우선순위와 선택 근거, 자원 고갈 예상, 증거 경로, 조치 담당자·시각, 보낸 신호와 주체, 변경 설정, 새 PID, 업무 완료·관제 Before/After, 재발 여부, 후속 코드 수정 담당자를 포함한다. 종료 전 총괄 담당자가 누락과 복구 기준 충족을 확인한다.

[Analysis] 정상 작업 로그는 FCFS와 부합하는 순차 처리 패턴

1. 로그 관찰 개요

보너스 과제로 CPU After의 정상 실행(MEMORY_LIMIT=512, CPU_MAX_OCCUPY=40, MULTI_THREAD_ENABLE=false)을 분석했다. 대상은 애플리케이션의 작업 실행 순서이며, 로그만으로 Linux 커널의 스케줄러 알고리즘을 단정하지 않는다.

2. 증거 자료

2026-09-05 KST 원본 앱 로그 (../evidence/cpu/after/app.log)의 등록 및 시작·완료 순서는 다음과 같다. 같은 로그의 중간 진행률 40/60/80%도 각 작업 안에 연속 배치되어 있다.

작업	시작 시각	완료 시각	시작→완료
Thread-A	15:49:44.987	15:49:45.194	207ms
Thread-B	15:49:45.245	15:49:45.451	206ms
Thread-C	15:49:45.502	15:49:45.709	207ms

등록 로그는 **Registered Tasks: ['Thread-A', 'Thread-B', 'Thread-C']**였다. A 완료 후 B 시작까지 51ms, B 완료 후 C 시작까지 51ms이다. 15:49:45.760에 **All tasks completed.**가 기록되었다.

3. 패턴 분석 및 결론

작업이 등록된 A→B→C 순서대로 각각 100%까지 끝난 뒤 다음 작업이 시작되었다. 주어진 FCFS / Round-Robin / Priority 중에서는 FCFS와 부합하는 비선점 순차 처리가 가장 자연스러운 추론이다. 관측한 로그에는 미완료 작업 사이를 순환하는 Round-Robin 패턴이 없다. 우선순위 값이나 순서 역전 증거도 없다.

다만 같은 우선순위 작업들의 실행이나 단순 순차 호출도 같은 로그를 만들 수 있다. 한 실행의 출력만으로 내부 알고리즘이 FCFS로 구현되어 있다고 확정할 수 없다. 약 51ms의 로그 간격은 의도적 지연이나 작업 시간이 포함된 값일 수 있으므로 OS time quantum으로 해석하지 않는다.

ps 원본 (../evidence/cpu/after/ps.txt)에서 실제 LWP의 CLS는 TS, NI는 10이었다. 이 증거는 앱의 A/B/C 실행 순서와 커널 스케줄링 정책이 서로 다른 충위임을 보여준다. **Linux 스케줄러 공식 문서**는 실행 가능한 task의 CPU 배분을 설명하며, 애플리케이션 로그의 순차 출력 자체가 커널 FCFS 또는 SCHED_FIFO의 증거는 아니다.

4. 장단점과 적합한 서비스

FCFS 방식은 순서가 명확하고 구현이 단순해 작업 순서가 중요한 배치·직렬 처리에 적합하다. 긴 작업이 앞에 오면 짧은 작업까지 기다리는 **convoy effect**가 발생하므로 응답 지연이 중요한 웹 요청 처리를 모두 하나의 FCFS 대기열로 직렬화하는 데는 불리하다. 그런 서비스에는 작업 분리, 제한된 동시성, 공정한 시간 배분이나 짧은 작업을 별도 처리하는 설계를 고려할 수 있다. 이는 관측된 패턴에 대한 설계적 해석이며 실측 성능 비교 결과는 아니다.

증거 인덱스와 재현 안내

GitHub Issue 형식의 Markdown 원문은 reports/oom.md, reports/cpu.md, reports/deadlock.md이다. 보너스 원문은 reports/scheduling.md에 있다. FAIL #18의 대응 절차는 reports/incident-runbook.md, FAIL #15의 추가 직접 증거는 evidence/deadlock/diagnostic-01에 있다.

정식 비교: oom/before, oom/after, cpu/before-02, cpu/after, deadlock/before, deadlock/after. 각 폴더의 app.log, monitor-PID.log, ps.txt, top.txt, settings.json, result.json, preflight.txt, postflight.txt를 보존했다.

탐색 2회는 일반 리다이렉션으로 마지막 print가 유실될 수 있어, 정식 실험에서는 PTY로 표준 출력을 수집했다. CRLF만 LF로 정규화했고 앱 자체 로그도 별도 복사했다. 50ms 샘플은 CPU 정식 사례에 추가했다.

생존 시간은 실행 시작부터 부모 회수와 관측기 정리까지의 monotonic 시간이며 작은 계속 오버헤드가 포함된다. JSON/CSV/ps 헤더는 UTC, 앱·monitor는 KST이다. RSS_KB는 KiB, 1코어 CPU=100% 기준이다.

재현: Docker 환경에서 bash scripts/run_experiments.sh -rerun-01을 실행한다. 실행기는 기존 증거를 덮어쓰지 않는다. 폴더·키·권한·환경변수 준비와 비root 검증을 포함한다.

바이너리 SHA-256

7e0a19cfa80ece6b547a5008273661f0d4d71e526e96b51e0d0f341dd1bb3e40

원본 무결성 및 제출 범위

첨부 evidence.zip 안의 SHA256SUMS로 원본 로그를 검증할 수 있다. 이 PDF는 평가 FAIL #15-#18을 보완한 제출용 보고서이며 공개 저장소 NiceTry3675/B1-2에 함께 제공한다. 검증 기록과 실행·PDF 생성 스크립트는 프로젝트에 함께 제공한다.